UNIVERSIDAD DE ALCALÁ
DEPARTAMENTO DE AUTOMÁTICA
ÁREA DE ARQUITECTURA Y TECNOLOGÍA DE COMPUTADORES
ASIGNATURA DE **PROGRAMACIÓN AVANZADA**

Universidad de Alcalá

Práctica 1

Instrucciones:

Se podrá utilizar un "`Jupyter Notebook`" y crear cada ejercicio en un bloque de código con un comentario que enuncie el número del ejercicio.

Todas las funciones deberán contener anotaciones, `docstring` y seguir la convención de nombres explicada en clase de teoría.

Cada ejercicio deberá incluir una función `main()` que pruebe las funciones con los ejemplos del enunciado y con algún caso adicional.

Objetivos:

Introducción al lenguaje de programación Python: bucles, funciones, cadenas, listas y tipos de datos básicos.

Ejercicio 1

Escriba un programa que pida al usuario una cantidad de segundos y la descomponga en días, horas, minutos y segundos. Debe incluir las siguientes funciones (documentadas y con anotaciones):

1. `is_integer(text)`: recibe una cadena y devuelve `True` si representa un número entero (positivo, negativo o cero) y `False` en caso contrario. Utilice `try`/`except`. Debe aceptar "0" y "-5", y rechazar "3.14", "Hola" y "True". Se aceptan los mismos formatos que admite `int()`, por lo que "+5" o " 42 " (con espacios alrededor) también son válidos.
2. `seconds_to_dhms(total_seconds)`: recibe un entero no negativo y devuelve una tupla `(días, horas, minutos, segundos)`. Utilice los operadores `//` y `%`.
3. `main()`: solicita la cantidad de segundos, repitiendo la petición mientras la entrada no sea un entero no negativo, y muestra el resultado.

Pruebe el programa, al menos, con los valores 0, 59, 60, 3600 y 86400.

Ejemplo de salida:

```
Introduzca los segundos: hola
Entrada no válida. Introduzca un entero no negativo.
Introduzca los segundos: -5
Entrada no válida. Introduzca un entero no negativo.
Introduzca los segundos: 93784
93784 segundos son 1 d, 2 h, 3 min y 4 s.
```

Ejercicio 2

El calendario gregoriano (1582) establece que un año es bisiesto si es divisible entre 4, salvo que sea divisible entre 100; sin embargo, si además es divisible entre 400, también es bisiesto. Escriba un programa con las siguientes funciones:

1. `is_leap_year(year)`: devuelve `True` si el año es bisiesto y `False` en caso contrario. Debe resolverse con una única expresión booleana, sin utilizar `if`. Los booleanos se pueden operar como el resto de los tipos.
2. `leap_years_between(start, end)`: devuelve la lista de los años bisiestos comprendidos entre `start` y `end`, ambos incluidos. Utilice un bucle `for` y reutilice `is_leap_year()`.
3. `explain_leap_year(year)`: devuelve una cadena que indica si el año es bisiesto y por qué (véase el ejemplo).

La función `main()` pedirá un año al usuario. Si es anterior a 1582, mostrará que el calendario gregoriano aún no existía; en otro caso, mostrará la explicación. Pruebe, al menos, con los años 1900, 2000, 2023 y 2024. Las funciones `is_leap_year()` y `leap_years_between()` aplican la regla a cualquier año: la comprobación del año 1582 se realiza únicamente en `main()`.

Ejemplo de salida de `main()`:

```
Año: 1900
1900 no es bisiesto: es divisible entre 100 pero no entre 400.
Año: 2000
2000 es bisiesto: es divisible entre 400.
Año: 2024
2024 es bisiesto: es divisible entre 4 y no entre 100.
Año: 2023
2023 no es bisiesto: no es divisible entre 4.
Año: 1500
El calendario gregoriano se estableció en 1582.
```

Ejemplo de uso de `leap_years_between()`:

```
>>> leap_years_between(1896, 1912)
[1896, 1904, 1908, 1912]
```

Ejercicio 3

Escriba una función llamada `process_text()` que recibe una cadena de texto y una lista de palabras a reemplazar. Esta función deberá:

- Proporcionar valores predeterminados para la cadena de texto y la lista de palabras que se reemplazarán (en el ejemplo, la cadena `"This is a default text with Python and amazing words."` y la lista `["Python", "amazing"]`).
- Convertir la cadena a minúsculas.
- Eliminar los espacios en blanco iniciales y finales.
- Reemplazar con asteriscos (*) cualquier palabra de la cadena de texto que coincida con un elemento de la lista de palabras. Se consideran palabras las subcadenas separadas por espacios (`split()`); la comparación no distingue mayúsculas de minúsculas, cada palabra se sustituye por tantos asteriscos como letras tiene y no es necesario tratar los signos de puntuación.
- Devolver la cadena procesada y la cantidad de palabras reemplazadas.

Llame a `process_text()` desde una función principal.

```
# Ejemplo de salida con valores por defecto. Reemplazar: Python amazing
Processed text: "this is a default text with ****** and ******* words."
Palabras reemplazadas: 2
```

Ejercicio 4

El cifrado César sustituye cada letra de un texto por la que se encuentra `shift` posiciones más adelante en el alfabeto, volviendo al principio cuando se llega a la "z". El resto de caracteres (espacios, dígitos, signos de puntuación…) no se modifican, y las mayúsculas siguen siendo mayúsculas. Considere únicamente las letras del alfabeto inglés (a–z y A–Z). Escriba las siguientes funciones:

1. `shift_char(char, shift)`: desplaza un único carácter; si no es una letra, lo devuelve sin cambios. Utilice `ord()` y `chr()`; en el código no deben aparecer los códigos numéricos de la tabla ASCII (use, por ejemplo, `ord('a')`).
2. `encrypt(text, shift)`: cifra un texto recorriéndolo carácter a carácter y devuelve el resultado.
3. `decrypt(text, shift)`: descifra un texto cifrado con el mismo desplazamiento, reutilizando `encrypt()`.
4. `brute_force(cipher_text)`: devuelve una lista con los 26 posibles descifrados del texto, uno por cada desplazamiento de 0 a 25.

El desplazamiento es un entero que puede ser negativo o mayor que 26, y se interpreta módulo 26 (por ejemplo, desplazar 29 posiciones equivale a desplazar 3, y desplazar -3 equivale a desplazar 23). Llame a las funciones desde `main()`.

Ejemplo de uso:

```
>>> encrypt("Hola, Mundo!", 3)
'Krod, Pxqgr!'
>>> decrypt("Krod, Pxqgr!", 3)
'Hola, Mundo!'
>>> encrypt("xyz", 3)
'abc'
>>> encrypt("abc", -3)
'xyz'
>>> brute_force("Krod, Pxqgr!")[3]
'Hola, Mundo!'
```

Ejercicio 5

Una frase es un palíndromo si se lee igual de izquierda a derecha que de derecha a izquierda, ignorando espacios, signos de puntuación, mayúsculas y tildes (por ejemplo, "Anita lava la tina"). Escriba las siguientes funciones:

1. `normalize(text)`: devuelve el texto en minúsculas, sustituyendo las vocales con tilde o diéresis por la misma vocal sin ella (á→a, é→e, í→i, ó→o, ú→u, ü→u) y conservando únicamente letras y dígitos. Las demás letras (como la "ñ") se conservan, solo pasadas a minúsculas. Puede apoyarse en `str.lower()`, `str.replace()` y `str.isalnum()`.
2. `is_palindrome(text)`: devuelve `True` si el texto es un palíndromo. Utilice "slicing" con paso -1 para invertir la cadena normalizada.
3. `find_palindromes(sentences)`: recibe una lista de frases y devuelve una nueva lista con las que son palíndromos.
4. `palindrome_words(text)`: recibe una frase sin signos de puntuación y devuelve la lista de sus palabras palíndromas de al menos 3 letras, tal y como aparecen escritas (utilice `split()`).

Pruebe las funciones desde `main()` con la siguiente lista de frases y con alguna más de su elección:

```
sentences = [
    "Anita lava la tina",
    "Dábale arroz a la zorra el abad",
    "Esto no es un palíndromo",
    "¿Acaso hubo búhos acá?",
]
```

Ejemplo de salida:

```
>>> for phrase in find_palindromes(sentences):
...     print(phrase)
Anita lava la tina
Dábale arroz a la zorra el abad
¿Acaso hubo búhos acá?
>>> palindrome_words("Ana vio un oso en el ojo de Anita")
['Ana', 'oso', 'ojo']
```

Ejercicio 6

Escriba un programa que realice un análisis de números primos. Debe incluir las siguientes funciones (que deberán estar documentadas):

1. `is_prime(n)`: comprueba si un número `n` es primo (los números menores que 2 no lo son).
2. `prime_list(limit)`: genera una lista de números primos hasta un límite `limit` (incluido). Si `limit` es menor que 2, devuelve una lista vacía.
3. `check_palindrome(primes)`: busca palíndromos en una lista de números primos (se leen igual de adelante hacia atrás).
4. `categorize_prime(prime)`: clasifica los números primos en "pequeños" (menores que 10), "medianos" (de 10 a 99) o "grandes" (100 o más).

La función `main()` se utiliza para probar las funciones del programa. Puedes probar con diferentes números y límites.

Ejercicio 7

Escriba un programa con las siguientes funciones sobre listas de números enteros. No se permite utilizar `sorted()`, `list.sort()`, `set()` ni el módulo `statistics`: el objetivo es practicar el recorrido de listas con bucles. Ninguna función debe modificar la lista recibida como argumento.

1. `remove_duplicates(numbers)`: devuelve una nueva lista sin elementos repetidos, conservando el orden de la primera aparición de cada valor.
2. `rotate_left(numbers, k)`: devuelve una nueva lista con los elementos rotados `k` posiciones hacia la izquierda. Debe funcionar si `k` es mayor que la longitud de la lista, si es negativo (rota hacia la derecha), si es 0 (devuelve una copia igual de la lista) o si la lista está vacía.
3. `split_even_odd(numbers)`: devuelve una tupla con dos listas: los números pares y los impares, en su orden original.
4. `merge_sorted(a, b)`: recibe dos listas ya ordenadas de menor a mayor y devuelve una única lista ordenada. Recorra ambas listas a la vez con dos índices, sin ordenar el resultado final.

5. `second_largest(numbers)`: devuelve el segundo valor más grande (distinto del máximo) recorriendo la lista una sola vez; si no existe (lista vacía, con un solo elemento o con todos los elementos iguales), devuelve `None`.

Pruebe todas las funciones desde `main()`.

Ejemplo de uso:

```
>>> remove_duplicates([3, 1, 3, 2, 1])
[3, 1, 2]
>>> rotate_left([1, 2, 3, 4, 5], 2)
[3, 4, 5, 1, 2]
>>> rotate_left([1, 2, 3, 4, 5], 7)
[3, 4, 5, 1, 2]
>>> rotate_left([1, 2, 3, 4, 5], -1)
[5, 1, 2, 3, 4]
>>> rotate_left([1, 2, 3, 4, 5], 0)
[1, 2, 3, 4, 5]
>>> split_even_odd([1, 2, 3, 4, 5, 6])
([2, 4, 6], [1, 3, 5])
>>> merge_sorted([1, 4, 9], [2, 3, 10, 11])
[1, 2, 3, 4, 9, 10, 11]
>>> second_largest([4, 9, 2, 9, 7])
7
>>> print(second_largest([5, 5]))
None
```

Ejercicio 8

Escribe un programa que lea una lista de números enteros y realice las siguientes operaciones utilizando un menú que se repetirá tantas veces como sea necesario hasta que el usuario presione la opción de "salir". Defina funciones para cada opción y utilice una función `main()`.

1. Calcula la suma de todos los números.
2. Calcula la suma de los números positivos y la suma de los números negativos por separado.
3. Encuentra el número máximo y el mínimo de la lista.
4. Comprueba si todos los números de la lista son únicos.
5. Utiliza el operador condicional para devolver un mensaje que indique si la lista contiene más números positivos o negativos.

Tenga en cuenta los casos límite: si la lista está vacía, las opciones que no tengan resultado (el máximo y el mínimo) deben mostrar un mensaje en lugar de producir un error; una lista vacía se considera formada por elementos únicos; la suma de un grupo sin elementos es 0; el 0 no cuenta como positivo ni como negativo; y si hay tantos positivos como negativos, el mensaje de la última opción debe indicarlo.

Ejercicio 9

Un polinomio puede representarse con una lista de coeficientes en la que la posición indica el exponente. Por ejemplo, la lista `[4.0, 1.0, 0.0, 2.0]` representa el polinomio $4 + x + 0x^2 + 2x^3$, que se escribe `2.0x**3 + x + 4.0`. Los coeficientes pueden ser de tipo `int` o `float`. Escriba las siguientes funciones:

1. `format_monomial(coef, exp)`: devuelve el monomio como cadena, con su signo explícito, aplicando las reglas indicadas más abajo.
2. `format_polynomial(coefs)`: devuelve el polinomio completo, con los términos ordenados de mayor a menor exponente y omitiendo los de coeficiente 0. El primer término no lleva el signo "+" delante y los demás se unen con los signos "+" o "-" rodeados de espacios (véase el ejemplo). Si todos los coeficientes son 0 (o la lista está vacía), devuelve "0". Reutilice `format_monomial()`.
3. `evaluate(coefs, x)`: devuelve el valor del polinomio para un valor `x` dado (0 si la lista está vacía).
4. `derivative(coefs)`: devuelve la lista de coeficientes del polinomio derivado; si el polinomio es constante o la lista está vacía, devuelve una lista vacía.

Reglas para escribir un monomio:

- Si el coeficiente es 0 no se escribe el monomio (se devuelve una cadena vacía).
- Si el coeficiente es 1 o -1 y el exponente es mayor que 0, no se escribe el 1: solo el signo.
- Si el exponente es 1 no se escribe el exponente.
- Si el exponente es 0 no se escribe la `x`, solo el coeficiente.
- Advierta que Python no escribe el "+" en los números positivos: hay que añadirlo expresamente.

Ejemplo de uso:

```
>>> format_monomial(2.0, 3)
'+2.0x**3'
>>> format_monomial(1.0, 2)
'+x**2'
>>> format_monomial(-1.0, 1)
'-x'
>>> format_monomial(4.0, 0)
'+4.0'
>>> format_monomial(0.0, 5)
''
>>> format_polynomial([4.0, 1.0, 0.0, 2.0])
'2.0x**3 + x + 4.0'
>>> format_polynomial([0.0, -1.0, 3.0])
'3.0x**2 - x'
>>> format_polynomial([-2.5, 0.0, -1.0])
'-x**2 - 2.5'
>>> format_polynomial([0.0, 0.0])
'0'
>>> evaluate([4.0, 1.0, 0.0, 2.0], 2)
22.0
>>> derivative([4.0, 1.0, 0.0, 2.0])
[1.0, 0.0, 6.0]
```

Ejercicio 10

Escriba un programa que lea una lista de cadenas desde teclado y realice las siguientes operaciones utilizando un menú que se repetirá tantas veces como sea necesario hasta que el usuario presione la opción de "salir". Defina funciones para cada opción y utilice una función `main()`.

1. Imprima cada cadena que comience con una letra pedida al usuario.
2. Cuente cuántas cadenas contienen una subcadena pedida al usuario.
3. Encuentre la cadena más larga y la más corta de la lista (en caso de empate, la primera que aparezca; si la lista está vacía, se mostrará un mensaje).
4. Utilice el operador `is` para verificar si dos cadenas específicas de la lista son en realidad el mismo objeto.
5. Utilice un bucle `for...else` para verificar si alguna cadena de la lista tiene una longitud mayor que 10. Si no es así, imprima un mensaje que indique que ninguna cadena tiene más de 10 caracteres.

Ejercicio 11

En un hospital se quiere compartir informes médicos con estudiantes, pero antes es necesario eliminar los datos sensibles de los pacientes y médicos.

Para ello, se pide escribir un programa en Python que use expresiones regulares (`re`) para procesar un informe médico simulado (guardado en una cadena). Las fechas en los

informes aparecen en formato dd/mm/aaaa. Para detectarlas, utilice el método `re.compile()` con una expresión regular que reconozca tanto días y meses de una o dos cifras, como años de cuatro cifras.

Debes sustituir:

- Fecha de nacimiento (línea que empieza con `Fecha de nacimiento:`) → por una línea `Edad: N`, siendo N los años que tenía el paciente en la fecha de la consulta (la fecha que aparece en la línea del médico). Si en esa fecha aún no había cumplido años, se resta uno.
- Resto de fechas en formato dd/mm/aaaa → por la palabra "FECHA"
- Nombre y apellidos del paciente (línea que empieza con `Nombre:`) → por "PACIENTE", conservando la etiqueta `Nombre:`
- Nombre del médico (`Dr.` o `Dra.` y el nombre que le sigue hasta el final de la línea) → por "MEDICO"; el resto de la línea se conserva

Atención al orden de las sustituciones: la fecha de nacimiento debe procesarse antes que el resto de las fechas; si primero se sustituyen todas las fechas por "FECHA", ya no es posible calcular la edad.

Texto de ejemplo:

```
informe = """Informe clínico de Urgencias
Nombre: Juan Pérez López
Género: Hombre
Fecha de nacimiento: 12/05/1980
Motivo de consulta: dolor abdominal
12/05/2023 Dr. Ramírez
Tratamiento: reposo y analgésicos"""
```

Resultado esperado:

```
Informe clínico de Urgencias
Nombre: PACIENTE
Género: Hombre
Edad: 43
Motivo de consulta: dolor abdominal
FECHA MEDICO
Tratamiento: reposo y analgésicos
```

Ejercicio 12

Dos palabras son anagramas si tienen exactamente las mismas letras, en otro orden (por ejemplo, “roma” y “amor”). Escriba un programa con las siguientes funciones (documentadas y con anotaciones):

1. `is_anagram(word1, word2)`: devuelve `True` si las dos cadenas son anagramas, ignorando mayúsculas y espacios. Puede apoyarse en `sorted()`, `str.lower()`, `str.replace()` y `"".join()`.
2. `group_anagrams(words)`: recibe una lista de palabras y devuelve una lista de listas. Cada sublista agrupa las palabras que son anagramas entre sí. Los grupos y las palabras dentro de cada grupo se mantienen en el orden de primera aparición, y una palabra sin anagramas forma un grupo de un solo elemento. Reutilice `is_anagram()`.
3. `largest_group(groups)`: recibe el resultado de `group_anagrams()` y devuelve el grupo con más palabras (el primero en caso de empate). Si la lista está vacía, devuelve una lista vacía.
4. `find_anagrams(word, words)`: devuelve la lista de las palabras de `words` que son anagramas de `word`, excluyendo todas las palabras de `words` que sean iguales a `word` (sin distinguir mayúsculas y minúsculas). Los demás anagramas se devuelven tantas veces como aparezcan en la lista.

La función `main()` probará las funciones con la lista de palabras del ejemplo y con alguna más de su elección.

Ejemplo de uso:

```
words = ["roma", "amor", "pelo", "mora", "casa", "lope", "ramo"]

>>> is_anagram("Roma", "amor")
True
>>> is_anagram("casa", "caso")
False
>>> is_anagram("dormitory", "dirty room")
True
>>> group_anagrams(words)
[['roma', 'amor', 'mora', 'ramo'], ['pelo', 'lope'], ['casa']]
>>> largest_group(group_anagrams(words))
['roma', 'amor', 'mora', 'ramo']
>>> find_anagrams("amor", words)
['roma', 'mora', 'ramo']
>>> find_anagrams("amor", ["amor", "roma", "AMOR", "roma"])
['roma', 'roma']
>>> find_anagrams("sol", words)
[]
```